

Decision Tree Report

Introduction

This project focuses on implementing a decision tree classifier in C++ for a dataset consisting of ten binary features and a class label with three possible values. The aim of the task is to construct a model that learns a set of decision rules from training data and applies them to predict the class labels of unseen test examples.

The performance of the model is evaluated by measuring classification accuracy on a separate test dataset, while checking the number of decision rules used in the tree.

Implementation

The dataset is read from text files (training.dat and testing.dat). Each row contains ten binary feature values followed by a class label. These are stored using vectors, where one structure holds feature values and another stores corresponding labels.

The decision tree is implemented using a single Node class. Each node can represent either:

- a leaf node, which stores a predicted class label, or
- a rule node, which stores a feature index used to split the data, along with pointers to left and right child nodes

Leaf nodes represent the final classification decision, while rule nodes guide the traversal of the tree based on feature values.

Measuring Purity: To determine the best way to split the data, the **Gini impurity** measure is used. This metric evaluates how mixed the class labels are within a node.

A perfectly pure node that contains only one class has a Gini value of 0, while more mixed nodes have higher values.

For each candidate split:

- the dataset is divided into two groups based on a feature
- the Gini impurity is calculated for both groups
- a weighted average is taken
- the improvement is computed relative to the parent node

The best split is the one that maximizes this reduction in impurity.

Using a tree growth strategy:

Instead of growing the tree layer by layer, a best first approach is used.

The process works as follows:

1. The entire training dataset is placed in a single root node
2. All possible feature splits are evaluated for that node
3. The split with the highest impurity reduction is selected
4. The node is converted into a rule node and split into two child nodes
5. Each child node is then evaluated and added to a priority queue
6. The node with the highest potential improvement is always chosen next

This continues until a predefined number of rule nodes has been created.

Prediction:

To classify a new data point, the model starts at the root and follows decision rules:

- at each rule node, a feature value is checked
- depending on whether it is 0 or 1, the left or right branch is taken
- this continues until a leaf node is reached

The class label stored in the leaf node is returned as the prediction.

Results

The classifier was tested using the provided testing dataset while increasing the number of rules in the tree.

The results showed that:

- with very few rules, accuracy is relatively low (around 50%)
- adding more rules improves performance significantly at first
- after a certain point, accuracy stabilises around 63–64%
- further increasing the number of rules does not lead to meaningful improvement

This indicates that only a limited number of features provide strong predictive power.

Use of AI Tools

During the implementation, I encountered issues when converting leaf nodes into rule nodes while using the priority queue approach. In particular, I found that after splitting a node, the tree structure was not updating correctly, which caused the model to produce constant predictions regardless of the number of rules. To resolve this, I used an AI assistant to help understand how to correctly modify nodes in place without breaking the tree structure. I asked the following prompt:

“In my C++ decision tree, I am using a priority queue to split leaf nodes, but after splitting, the tree does not update correctly and accuracy stays constant. How should I properly convert a leaf node into a rule node while keeping the tree structure valid?”

The response suggested storing additional information within each node and modifying the node directly (rather than replacing it externally), including updating its type, assigning child nodes, and preserving its position in the tree. I then implemented a corrected version where each node contains both structural and data information, and verified the fix by observing that accuracy improved as the number of rules increased.

Conclusion

A decision tree classifier was successfully implemented using a best first splitting strategy. The model achieved reasonable classification performance on the test dataset, with accuracy stabilising around 63–64%. The results demonstrate that decision trees can effectively learn patterns from binary feature data, especially when combined with a strategy that prioritises the most informative splits.

Rules 1	Accuracy: 50.3%
Rules 2	Accuracy: 61.1%
Rules 3	Accuracy: 60.6%
Rules 4	Accuracy: 60.6%
Rules 5	Accuracy: 60.6%
Rules 6	Accuracy: 63.8%
Rules 7	Accuracy: 64.2%
Rules 8	Accuracy: 64.2%
Rules 9	Accuracy: 64.2%
Rules 10	Accuracy: 64.9%
Rules 11	Accuracy: 64.9%
Rules 12	Accuracy: 64.9%
Rules 13	Accuracy: 64.6%
Rules 14	Accuracy: 64%
Rules 15	Accuracy: 64.1%
Rules 16	Accuracy: 64.1%
Rules 17	Accuracy: 64.1%
Rules 18	Accuracy: 64.1%
Rules 19	Accuracy: 64.7%
Rules 20	Accuracy: 64.2%